Fabula: AI-Generated Mystery Game

Deliverable 1: Problem Definition and Data Foundations

Hassan Nawaz (24L-2610) — Muhammad Ibrahim (24L-2602) — Shanzay Shahid (22L-7451)
BS Data Science, FAST-NUCES Lahore

October 2026

Abstract

Fabula is an AI-powered mystery game designed to generate diverse, replayable investigative experiences while maintaining narrative consistency and controlled information disclosure. Traditional hand-authored mysteries require considerable development effort, while rule-based generation can produce repetitive or incoherent scenarios. Unconstrained large language models (LLMs) may also introduce contradictions or reveal information that non-player characters (NPCs) should keep secret. Fabula addresses these challenges through a hybrid architecture combining AI-based narrative generation with deterministic game logic and structured validation. This report presents the problem definition, target audience, rationale for using AI, and the initial data acquisition and preprocessing pipeline. The current dataset comprises 300 synthetically generated mystery scenarios, balanced across theft, disappearance, and murder cases. The preprocessing pipeline applies schema and semantic validation, duplicate detection, feature engineering, categorical encoding, and numerical scaling. Exploratory data analysis examines dataset composition and information-disclosure policies. These foundations support future expansion into natural-language training examples, model fine-tuning, and systematic evaluation of NPC consistency and secret-keeping behaviour.

Contents

1 Introduction and Problem Statement **1**

1.1 Problem Statement 1

1.1.1 The Content Bottleneck in AI-Driven Mystery Games 1

1.2 Proposed Solution: Fabula 1

2 Target Audience and Objectives **3**

2.1 Target Audience 3

2.1.1 Players of Short Mystery and Puzzle Games 3

2.1.2 Indie Game Developers and Students 3

2.2 Research Gap 3

2.3 Project Objectives 4

3 Justification for Using Artificial Intelligence **5**

3.1 Why AI Is Appropriate for Fabula 5

3.2 Hybrid AI and Deterministic Architecture 5

3.3 Role of Fine-Tuning 6

4 Data Acquisition and Dataset Description **7**

4.1 Data Collection Approach 7

4.2 Dataset Composition 7

4.3 Collection Evidence and Reproducibility 8

5 Data Cleaning and Validation **9**

5.1 Purpose of Preprocessing 9

5.2 Schema Validation 9

5.3 Semantic Validation 9

5.4 Duplicate Detection 10

5.5 Handling Missing Values and Out-of-Range Values 10

6 Feature Engineering and Preprocessing **11**

6.1 Feature Extraction 11

6.2 Categorical Encoding 11
6.3 Numerical Scaling 11
6.4 Preprocessing Summary 12

7 Exploratory Data Analysis **13**
7.1 Purpose of EDA 13
7.2 Mystery Category Distribution 13
7.3 Disclosure Policy Distribution 14
7.4 Feature Correlation Analysis 14
7.5 EDA Findings and Limitations 15

8 Conclusion and Future Work **17**
8.1 Conclusion 17
8.2 Future Work 17

Chapter 1

Introduction and Problem Statement

1.1 Problem Statement

1.1.1 The Content Bottleneck in AI-Driven Mystery Games

Mystery and adventure games rely on coherent narratives, consistent timelines, interconnected clues, and characters with distinct knowledge and motivations. Creating such experiences is challenging because a mystery must remain logically solvable while preserving uncertainty for the player. Existing approaches face three major limitations.

First, **hand-authored content is expensive to produce and limited in replayability**. Developing a complete mystery requires writers to design the crime, suspects, alibis, clues, and narrative relationships. Once a player solves the case, the experience offers limited novelty unless additional content is created, increasing development time and cost.

Second, **relying entirely on large language models (LLMs) during gameplay introduces reliability and operational challenges**. Cloud-based models can incur recurring API costs and response latency. More importantly, unconstrained generation can produce contradictory statements, allow characters to claim knowledge they should not possess, or reveal the solution prematurely. Such failures undermine the investigative experience.

Third, **conventional procedural generation can struggle to maintain narrative consistency**. Template-based systems may generate varied scenarios but still produce conflicting timelines, weak clue relationships, or mysteries that are either unsolvable or too easy to resolve. Generating content alone is therefore insufficient; the resulting mystery must also satisfy structural and logical constraints.

These limitations motivate the need for a system capable of generating varied, coherent, and solvable mysteries while preserving character-specific knowledge and preventing unintended disclosure of the solution.

1.2 Proposed Solution: Fabula

Fabula addresses this challenge through a hybrid architecture that combines language-model-based content generation with deterministic game logic and structured validation. Rather than generating the entire mystery dynamically during play, the system is designed to generate the case before gameplay begins.

A locally executed language model of approximately 3–4 billion parameters is intended to

generate the mystery, setting, characters, and their private knowledge as structured JSON files. These outputs are subject to schema validation and cross-reference checks, with additional mechanisms intended to verify solvability and culprit uniqueness. Invalid outputs can be retried before a case is accepted.

During gameplay, a Python-based engine maintains the ground truth, including the actual solution, independently of NPC dialogue generation. Character movement is handled through schedules and A* pathfinding, while a bounded play session, clue journal, and accusation mechanism structure the player's investigation.

NPC dialogue is generated through live model APIs. To reduce accidental information leakage, each NPC's prompt is designed to use only that character's `identity.json` and `memory.json` files. The intended result is a knowledge boundary in which characters provide information consistent with their individual perspectives, while the culprit holds the true account and other characters possess partial observations or misleading personal information.

By separating narrative generation, game-state management, movement, and dialogue, Fabula aims to combine the creative flexibility of generative AI with the consistency and control of deterministic systems. This architecture provides a foundation for developing replayable mystery experiences while reducing contradictions, preserving investigative challenge, and limiting premature solution disclosure.

Chapter 2

Target Audience and Objectives

2.1 Target Audience

Fabula is intended to serve two primary user groups within its application and research objectives.

2.1.1 Players of Short Mystery and Puzzle Games

The primary application audience consists of casual mystery enthusiasts and players interested in AI-driven gaming who seek short, replayable investigative experiences. Fabula is designed around self-contained mystery sessions of approximately ten minutes, allowing players to investigate a case, interact with non-player characters (NPCs), collect clues, and identify the culprit without requiring prior knowledge of the story.

Its intended local-generation setup targets consumer-grade CPU hardware, while NPC conversations rely on live language-model APIs. This design aims to reduce hardware requirements while supporting variety across playthroughs.

2.1.2 Indie Game Developers and Students

Fabula also serves as a reference architecture for developers and students exploring generative AI in interactive applications. Its separation of pre-play content generation, deterministic game logic, and character-specific information provides a design pattern for building AI-driven games that are more structured, reproducible, and testable.

2.2 Research Gap

Although large language models have been used to power conversational agents and role-playing characters, evaluating whether these characters consistently respect their assigned knowledge and information-disclosure rules remains a challenge. Fabula focuses on five measurable aspects of NPC behaviour:

1. **Secret-keeping:** Whether an NPC conceals information it is instructed to hide or misrepresent.
2. **Knowledge boundaries:** Whether an NPC avoids claiming facts that are absent from its assigned knowledge state.
3. **Consistency:** Whether an NPC's responses remain consistent with its structured memory and established cover story across multiple interactions.

4. **Disclosure-policy adherence:** Whether an NPC follows the assigned disclosure categories: volunteer, if-asked, hide, and lie.
5. **Provider robustness:** Whether NPC behaviour remains consistent when different live-API model providers are used.

Fabula represents NPC knowledge as structured facts with explicit `source` and `disclosure` fields. This representation supports the development of scripted probe tests and fact-matching methods to assess character behaviour systematically. It also provides a basis for comparing base and fine-tuned models to investigate whether fine-tuning improves adherence to knowledge boundaries, disclosure policies, and narrative consistency.

2.3 Project Objectives

The project has the following objectives:

1. **Generate diverse mystery scenarios:** Develop a pipeline for producing structured mystery cases with settings, characters, clues, timelines, and character-specific knowledge.
2. **Validate generated content:** Apply schema and semantic checks to identify invalid or inconsistent scenarios before gameplay.
3. **Separate AI generation from game-state control:** Use a deterministic Python engine to manage ground truth, character movement, and investigation mechanics independently of generated dialogue.
4. **Enforce character-specific knowledge boundaries:** Restrict NPC dialogue context to the relevant character's identity and memory information to reduce unintended disclosure.
5. **Evaluate NPC behaviour systematically:** Establish measurable tests for secret-keeping, knowledge boundaries, consistency, disclosure-policy adherence, and robustness across model providers.
6. **Investigate the impact of fine-tuning:** Compare base and fine-tuned models, where available, to assess changes in NPC reliability and compliance with assigned behavioural constraints.

Together, these objectives aim to establish Fabula as both a replayable AI-driven mystery game and a testable framework for investigating reliable, knowledge-constrained NPC behaviour.

Chapter 3

Justification for Using Artificial Intelligence

3.1 Why AI Is Appropriate for Fabula

Fabula requires the generation of varied stories, character backgrounds, dialogue, and investigative scenarios. These tasks involve natural language, contextual relationships, and creative variation, making generative AI a suitable component of the proposed system.

A purely rule-based generator could produce structured cases from predefined templates, but maintaining variety across narratives and natural dialogue would require a large collection of manually designed rules and content. An LLM can generate more flexible narrative content, while validation mechanisms can constrain its outputs to the structures required by the game.

AI is particularly relevant to two parts of Fabula:

- **Pre-play narrative generation:** A language model can generate different settings, characters, motives, clues, and private knowledge for new cases.
- **NPC dialogue:** Language models can produce context-sensitive responses that make interactions with characters more natural than fixed dialogue trees.

However, generative models do not inherently guarantee logical consistency, correct information boundaries, or compliance with game rules. Their outputs therefore require constraints and independent validation.

3.2 Hybrid AI and Deterministic Architecture

Fabula combines generative AI with deterministic components rather than delegating every part of the game to a language model.

Table 3.1: Responsibilities of AI and deterministic components

Component	Primary responsibility
Local language model	Generate structured mystery content before gameplay.
Live-API language model	Generate dialogue using the relevant NPC's assigned context.
Python game engine	Maintain game state and authoritative ground truth.
Schedules and A*	Determine character movement and paths.
Schema and semantic validators	Check structural requirements and selected logical constraints.
Evaluation scripts	Assess NPC responses against expected facts and disclosure rules.

This division of responsibilities reduces reliance on probabilistic language generation for tasks that require predictable outcomes. For example, movement and the authoritative solution can be managed by program logic, while AI handles narrative variation and natural-language interaction.

3.3 Role of Fine-Tuning

Fine-tuning is intended to improve the model's ability to generate content in Fabula's required format and follow task-specific constraints. A fine-tuned model could be evaluated on its ability to produce valid structured scenarios and support NPC behaviour consistent with assigned information-disclosure policies.

The planned evaluation should compare a base model with a fine-tuned model using the same test cases and behavioural probes. Relevant measures include output validity, knowledge-boundary violations, secret leakage, consistency across turns, and adherence to disclosure policies.

Fine-tuning and its benefits should be treated as future work unless the training process and evaluation results have been completed. The current data foundation is intended to support that later stage.

Chapter 4

Data Acquisition and Dataset Description

4.1 Data Collection Approach

The current Fabula dataset is synthetically generated rather than collected through web scraping or downloaded from a third-party dataset. Python-based generation code produces structured mystery skeletons, while manifests and collection logs provide evidence about the resulting data.

The generation pipeline is designed to create structured scenarios containing elements such as mystery type, locations, non-player characters, clues, timelines, and character-specific knowledge. This structure makes it possible to validate the scenarios and derive numerical and categorical features for analysis.

The principal project locations relevant to this stage are:

- `src/fabula/training_data/`: generation code.
- `src/fabula/schemas/`: data schemas and validation definitions.
- `data/scripts/collect.py`: collection or generation workflow.
- `data/reports/collection_log.json`: collection execution evidence.
- `data/reports/dataset_manifest.json`: dataset metadata.
- `deliv1_docs/phase2_data_evidence.md`: summary of the data-generation and preprocessing evidence.

4.2 Dataset Composition

The current dataset contains 300 synthetic mystery skeletons. The scenarios are balanced across three mystery categories: theft, disappearance, and murder, with 100 scenarios in each category.

Table 4.1: Summary of the current synthetic dataset

Metric	Value
Total scenarios	300
Theft scenarios	100
Disappearance scenarios	100
Murder scenarios	100
Invalid scenarios removed during cleaning	0
Duplicate scenarios removed during cleaning	0
Final cleaned scenarios	300

The balanced class distribution provides equal representation of the three mystery categories in the current dataset. However, balance in category counts alone does not establish that all scenarios are equally complex, realistic, or suitable for model training. Those properties require separate evaluation.

4.3 Collection Evidence and Reproducibility

Because the dataset is synthetic, collection evidence consists primarily of the generation code, execution records, and dataset metadata rather than external source URLs. The collection log documents the generation run, while the dataset manifest records information about the produced dataset.

The raw skeleton dataset is stored at `data/raw/skeletons.jsonl` in the project workflow. This raw data path is gitignored, so the file may not be included in the repository checkout. The corresponding metadata and documentation should be used to describe the collection process.

A dataset hash recorded in the project evidence begins with `f52b2e1a` and ends with `5827a`. The complete hash and its meaning should be taken from the current dataset manifest when documenting reproducibility.

Chapter 5

Data Cleaning and Validation

5.1 Purpose of Preprocessing

Synthetic generation does not automatically guarantee that every scenario is logically valid. A generated scenario may satisfy the expected data structure while still containing conflicting timelines, disconnected locations, or inconsistencies between the culprit's account and the available evidence.

Fabula therefore applies schema validation and semantic checks before treating generated scenarios as clean data. These checks aim to identify structural errors and selected logical conflicts before feature extraction and analysis.

5.2 Schema Validation

The project uses structured schemas to define required fields, expected data types, permitted values, and constraints on scenario components. Schema validation helps prevent malformed records from entering later stages of the pipeline.

Required fields and strict schema definitions also reduce the likelihood of missing or unexpected fields. Where values are constrained to specific ranges, the schema can reject values that fall outside the permitted bounds.

This is different from statistical outlier detection. The current pipeline's bounded values and schema constraints should not be described as evidence of a separate statistical outlier-removal procedure unless such a procedure is implemented.

5.3 Semantic Validation

In addition to structural validation, the cleaning pipeline applies semantic checks to relationships within a scenario. The checks documented for the project include:

- Connectivity of the scenario's location graph.
- Conflicts involving character schedules and timelines.
- Validity of time windows.
- Consistency of the culprit's true account.
- Relationships between lies, cover stories, clues, and facts.
- Consistency of relevant evidence with the intended culprit.

These checks are intended to detect problems that a schema alone cannot identify. Their effectiveness depends on the specific rules implemented and the cases covered by the validators.

5.4 Duplicate Detection

The cleaning pipeline uses canonical JSON representations and SHA-256 hashing to identify duplicate scenarios. Canonicalization provides a consistent representation for hashing, allowing records with equivalent serialized content to be compared.

The recorded cleaning results are summarized in Table 5.1.

Table 5.1: Recorded cleaning results

Cleaning metric	**Count**
Input scenarios	300
Invalid scenarios removed	0
Duplicate scenarios removed	0
Output scenarios	300

The results indicate that all 300 input scenarios were retained in the recorded run. Zero removals do not, by themselves, prove that every possible logical error has been eliminated; they show that no records were rejected by the checks and duplicate-detection process in that run.

5.5 Handling Missing Values and Out-of-Range Values

The dataset is generated from structured schemas with required fields and strict validation. Consequently, missing required fields are intended to be prevented or rejected during generation and validation, rather than imputed statistically after collection.

Likewise, bounded schema rules restrict certain structural values. This approach is appropriate for enforcing domain constraints in synthetic data. Nevertheless, the report does not claim that mean or median imputation, a separate statistical outlier analysis, or outlier removal was performed, as these operations are not documented in the current pipeline evidence.

The cleaning report is available at `data/reports/cleaning_report.json`, while the implementation is in `data/scripts/clean.py`.

Chapter 6

Feature Engineering and Preprocessing

6.1 Feature Extraction

After validation, structured mystery scenarios can be represented as numerical and categorical features for analysis and potential future machine-learning experiments.

Fabula's feature-engineering script extracts 16 numerical structural features. These features describe selected properties of the scenarios and provide a tabular representation for exploratory analysis and later modelling.

Categorical information includes mystery type, tone, and difficulty. These fields represent discrete categories and require encoding before they can be used directly by many machine-learning algorithms.

6.2 Categorical Encoding

The categorical variables `mystery_type`, `tone`, and `difficulty` are one-hot encoded. The current configuration uses three categories for each variable.

One-hot encoding represents each category using a separate binary column. This allows a model to distinguish categories without imposing an artificial numerical ordering between them.

6.3 Numerical Scaling

The 16 numerical structural features are min-max scaled to the range from 0 to 1. For a feature with minimum value $x_{\min}$ and maximum value $x_{\max}$, the transformation is:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

For non-constant features, this transformation maps the observed minimum to 0 and the observed maximum to 1. If a feature is constant, the denominator is zero; such features require special handling by the implementation and should not be interpreted as varying measurements.

The processed feature outputs are recorded in `features.csv` and `features_scaled.csv`. The

documented output contains 25 columns in each representation. The feature-engineering implementation is located at `data/scripts/features.py`.

6.4 Preprocessing Summary

Table 6.1: Summary of preprocessing operations

Operation	**Purpose**
Schema validation	Check required fields, data types, and permitted structures.
Semantic validation	Identify selected logical conflicts within generated scenarios.
Canonical JSON hashing	Detect duplicate scenarios.
Feature extraction	Represent scenarios through 16 numerical structural features.
One-hot encoding	Represent mystery type, tone, and difficulty as categorical columns.
Min-max scaling	Scale numerical features to the range from 0 to 1.

This preprocessing pipeline produces structured data suitable for exploration and future modelling. The quality of the resulting features still depends on their definitions, variation across scenarios, and relevance to the behaviours that Fabula ultimately intends to evaluate.

Chapter 7

Exploratory Data Analysis

7.1 Purpose of EDA

Exploratory data analysis (EDA) is used to understand the composition and characteristics of the synthetic dataset before using it in subsequent experiments. The current analysis includes the distribution of scenario categories, the distribution of information-disclosure policies, and relationships between numerical features.

These visualizations describe the current dataset; they are not evidence of model performance or proof that NPCs consistently follow their assigned policies.

7.2 Mystery Category Distribution

Figure 7.1 presents the distribution of the generated mystery categories. The current dataset contains 100 theft scenarios, 100 disappearance scenarios, and 100 murder scenarios.

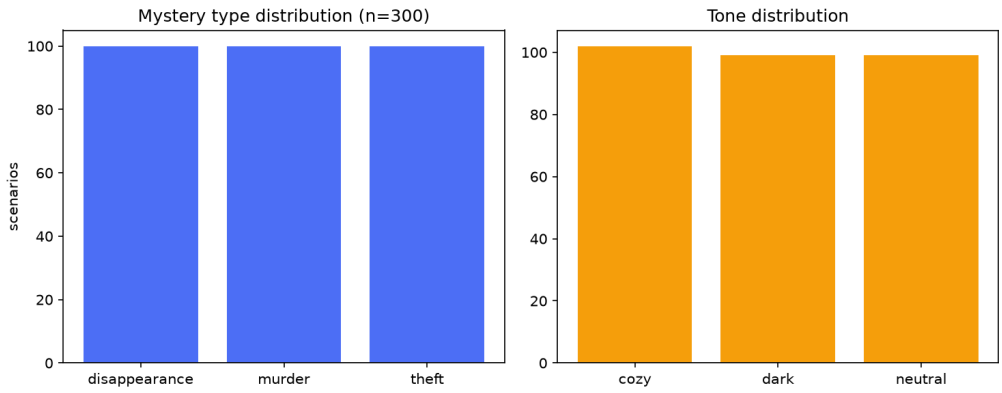


Figure 7.1: Distribution of mystery categories in the synthetic dataset.

The equal category counts indicate that the dataset is balanced with respect to mystery type. This balance is useful for initial comparisons across categories, although it does not establish equivalent narrative difficulty or logical complexity.

7.3 Disclosure Policy Distribution

NPC knowledge records use disclosure categories to specify how information should be handled. The current analysis records four categories: volunteer, if-asked, hide, and lie.

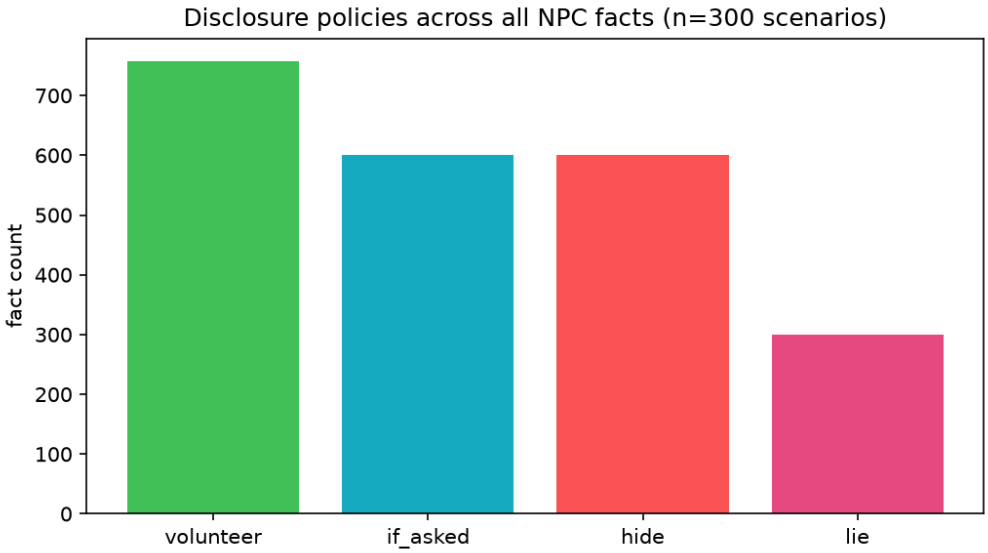


Figure 7.2: Distribution of disclosure policies across the recorded facts.

The recorded distribution contains 2,258 facts across the four categories, as summarized in Table 7.1.

Table 7.1: Recorded disclosure-policy counts

Disclosure category	Number of facts
Volunteer	758
If-asked	600
Hide	600
Lie	300
Total	2,258

The categories describe intended information-handling behaviour, not observed NPC performance. For example, the presence of facts labelled as hidden or misleading does not demonstrate that an NPC will successfully keep the information secret during a conversation. That requires separate behavioural testing.

7.4 Feature Correlation Analysis

Figure 7.3 presents the correlation analysis of numerical features. Correlation analysis can help identify relationships between variables and highlight features that may contain overlapping information.

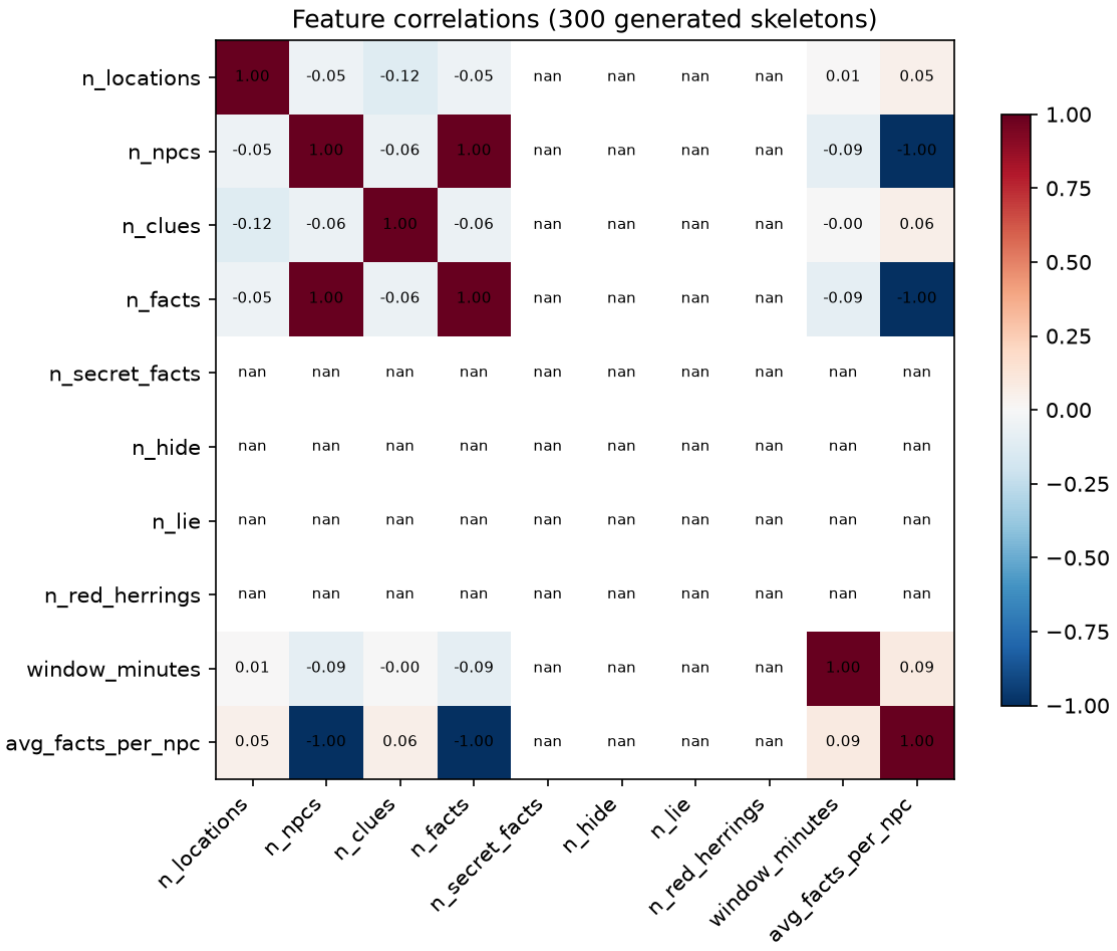


Figure 7.3: Correlation matrix of numerical structural features.

Some features may be strongly correlated because one is derived from another. A perfect negative correlation, for example, can occur when two variables encode complementary versions of the same quantity. Such relationships should be examined before using both features in a predictive model.

Some features are constant across all 300 skeletons. A constant feature has no variance, so its correlation with other features is undefined and may appear blank or as a missing value in the correlation matrix. This is an expected mathematical outcome rather than necessarily an error in the plotting process. Constant features may provide little information for distinguishing scenarios and should be reviewed during subsequent feature selection.

7.5 EDA Findings and Limitations

The current EDA supports three initial observations:

- The 300 scenarios are equally distributed across the three mystery categories.
- The recorded facts use four disclosure policies, with volunteer as the most frequent category in the current data.
- Feature relationships and constant columns require consideration before the features are used for modelling.

These findings describe the present synthetic dataset only. Further evaluation is required to establish scenario diversity, narrative quality, solvability, and the reliability of NPC information-disclosure behaviour. The current charts should not be interpreted as evidence that the intended model improvements have already been achieved.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This report has defined the problem addressed by Fabula and described the intended audience, project objectives, AI rationale, and initial data foundations. Fabula aims to combine generative language models with deterministic game logic to create replayable mystery experiences while controlling the information available to individual non-player characters.

The current data pipeline uses synthetic generation, structured schema validation, semantic checks, duplicate detection, feature extraction, categorical encoding, and numerical scaling. The dataset contains 300 mystery skeletons balanced across theft, disappearance, and murder. Initial EDA examines category balance, disclosure-policy counts, and relationships between numerical features.

These steps establish a structured foundation for further development. However, balanced data and successful preprocessing alone do not establish narrative solvability or reliable NPC behaviour. Those properties require explicit tests and evaluation during subsequent project stages.

8.2 Future Work

The next stages of Fabula are intended to include:

1. **Dataset expansion:** Expand the existing skeleton dataset into a larger collection of natural-language training examples, subject to the project's planned data-generation process.
2. **Model fine-tuning:** Fine-tune the intended local language model once the training examples, configuration, and computational requirements are ready.
3. **Behavioural evaluation:** Develop scripted probes to measure secret-keeping, knowledge-boundary violations, consistency across turns, and adherence to disclosure policies.
4. **Comparative experiments:** Where the relevant models and evaluation infrastructure are available, compare base and fine-tuned models under consistent testing conditions.
5. **Robustness testing:** Evaluate whether NPC behaviour remains consistent across supported live-API providers and different interaction scenarios.

The results of these stages will help determine whether Fabula's hybrid architecture can provide both varied narrative content and measurable control over NPC knowledge and disclosure.

Bibliography

[1] Z. Sprague et al., *MuSR: Testing the Limits of Chain-of-Thought with Multistep Soft Reasoning*, International Conference on Learning Representations (ICLR), 2024.

[2] M. Del and M. Fishel, *True Detective: A Deep Abductive Reasoning Benchmark Undoable for GPT-3 and Challenging for GPT-4*, Proceedings of *SEM, 2023.

[3] Pydantic, *Pydantic Documentation*. Available at: https://docs.pydantic.dev/

[4] Fabula Project Repository, *Source Code, Data Generation, Validation, and Project Documentation*. Available at: https://github.com/HassanNawaz14/Fabula